# RSCL — Algorithm 12: Hybrid SwarmChat: Learned Intent Routing, Dispatch, Follow-ups and Fallback

Reconfigurable Space Computing Lab (RSCL), California State Polytechnic University, Pomona (Cal Poly Pomona) · Director: Prof. Mohamed El-Hadedy

Date: 2026-11-12 · Document type: formal algorithm specification of the implemented code (HU-MA'AT drone-fleet simulation demo)
Source files described (content hash and state at document build time):

| | | | |
|---|---|---|---|
| hu-telepresence-gazebo/spaceops_v8/chat_intent.py | sha256 4db751eb3b05 | 2026-11-11 05:42 UTC | clean @ 10ee643 |
| hu-telepresence-gazebo/spaceops_v8/swarmchat_hybrid.py | sha256 26bcc6419064 | 2026-11-11 06:47 UTC | clean @ 10ee643 |
| hu-telepresence-gazebo/spaceops_v8/chat_llm_service.py | sha256 9c6cdcef5c67 | 2026-11-11 08:05 UTC | clean @ 10ee643 |
| hu-telepresence-gazebo/spaceops_v8/spaceops_server.py | sha256 1bdb818b2cfc | 2026-11-12 00:52 UTC | modified vs 983eede |

## 1. Purpose and context

SwarmChat lets visitors and operators talk to the five aircraft as personas in the SpaceOps dashboard. The design separates *understanding* (a learned intent router), *facts* (deterministic telemetry engine), *wording* (an optional small local instruct model, checked by Algorithm 13) and *authority* (none: chat never reaches a flight endpoint). This document specifies the router and the dispatch logic.

## 2. Inputs, outputs, data

- **Input:** message (1..350 chars), recipient ∈ {FLEET, Horus, Ra, Thoth, Anubis, Osiris}, a copied telemetry snapshot, recent events (35), online flag, per-aircraft health series (Algorithm 17).
- **Router output:** {intent, best, confidence, top3, latency_ms, source}; intent = `uncertain` if confidence < 0.55.
- **Intents (11):** social, banter, user_feelings, meta, joke, status, relation, history, followup, command, impossible.
- **State:** bounded conversation memory `turns` (deque, 12) and `last` = {intent, utterance, recipient, speakers, replies}; persona rotation counter.
- **Output:** list of chat messages, each with speaker, role, text, evidence and a `source` label (TELEMETRY_FACT, UNKNOWN, CHAT_LOCAL_LLM, RULES_PERSONA, SAFETY_SUPERVISOR, MISSION sources).

## 3. Algorithm

**Algorithm 12.1 Intent router (inference and training)**

Source: spaceops_v8/chat_intent.py: IntentRouter.features, IntentRouter.route, train

```
function FEATURES(texts)                                  // identical pooling to HU (Algorithm 1)
  h ← SmolLM2-135M(texts, max 64 tokens).last_hidden_state   // frozen, no gradients
  return concat(masked mean(h), masked max(h))
function ROUTE(text)
  if no checkpoint then return (uncertain, 0.0, ROUTER_UNAVAILABLE)
  p ← softmax(W · (FEATURES(text) − μ)/σ + b)          // single linear layer
  return (labels[argmax p] if max p ≥ 0.55 else uncertain, max p, top-3)
procedure TRAIN(seed 0)
  X ← FEATURES(train texts); μ, σ ← column mean, std + 1e-6;  Z ← (X − μ)/σ
  500 full-batch AdamW steps (lr 3e-3, weight decay 2e-2) on cross-entropy with label smoothing 0.05
     over dropout(Z, 0.25);  report held-out accuracy, confident accuracy/coverage at 0.55,
     command false positives/negatives;  save {labels, μ, σ, head, meta}
```

**Algorithm 12.2 HybridChat.ask (dispatch)**

Source: spaceops_v8/swarmchat_hybrid.py: HybridChat.ask, _mission_discussion, _dispatch, _facts, _history, _command, _social, _voice, _followup

```
function ASK(u, recipient, state, events, online, health)
  validate lengths and recipient;  with chat lock:
  t ← u without leading address/politeness ("team, can you ...")
  if PARSE-OBJECTIVE(t) succeeds then                // Algorithm 11 grammar
    plan ← Coordinator.plan(t) against the live supervisor (read-only dry runs)
    post its transcript (raw telemetry lines omitted; slot overrides merged into one line)
    post "Plan ... is STATUS. Chat never executes missions ..."; return
  r ← POST /route (2.5 s timeout) or (uncertain, ROUTER_UNAVAILABLE)
  if r is ROUTER_UNAVAILABLE or wording service health unknown then
    return deterministic SwarmChat.ask(u, ...)       // rules engine, never silent
  short ← ≤ 6 words;  leans ← starts with and|what about|how about|so|but|why|really|explain|tell me more|go on
  if last ≠ None and (r.intent = followup or (short and leans and r.intent ∈ {uncertain, meta, followup, social})) then
    return FOLLOWUP(u)
  return DISPATCH(r.intent, u)
function DISPATCH(intent, u)
  status, relation → deterministic facts (if the rule classifier misses it, a status line per addressed aircraft)
  history         → per speaker (≤ 2): if ≥ 4 fresh health samples: draft "largest tracking error ... now ..."
                    (+ first HOLD/FAULT/REFUSE event), else "I can't verify that ..."; VOICE(draft)
  command         → persona refusal + MA'AT note with a read-only preview from mission_intents.parse
                    (never dispatched; "Only the HU console ... can move an aircraft")
  otherwise       → SOCIAL: speakers (recipient; named ≤ 2; meta → Thoth; user_feelings → Horus, Osiris;
                    impossible → Ra, Anubis; social → 2 rotating; else 1 rotating); per speaker build
                    evidence (mode, fault, airborne, nearest, validated formations, latest acceptance line,
                    architecture facts, authorised manoeuvres) and VOICE(no draft) with a persona fallback;
                    impossible adds a MA'AT line "Chat cannot move aircraft"
function VOICE(speaker, u, evidence, draft, style, fallback)
  res ← POST /generate {persona, evidence, draft, last 8 turns, style, max_new_tokens 60, timeout 14 s} (16 s)
  if res.accepted and res.text then return (res.text, CHAT_LOCAL_LLM)
  return (fallback, RULES_PERSONA or the fact source)          // reason recorded as llm_fallback
function FOLLOWUP(u)
  if u has why|explain|mean|simpl|plain|elaborate|more|again|five and last has replies then
    VOICE(previous speaker, draft = previous reply, style explain, fallback "In short: " + previous)
  else targets ← named aircraft, or (others|rest|everyone|all) → aircraft that did not speak last
    if targets = ∅ then DISPATCH(uncertain, u)
    re-ask the previous utterance (names stripped) to the new targets with the previous intent
```

## 4. Flowchart and state machine

Figure 12.1 is the control flow of one call to `HybridChat.ask`, drawn from the code (branch order as implemented: input validation, mission-objective grammar, router, availability fallback, follow-up test, intent dispatch). Figure 12.2 is the only state the module keeps between messages: the record of the previous turn used for follow-ups.

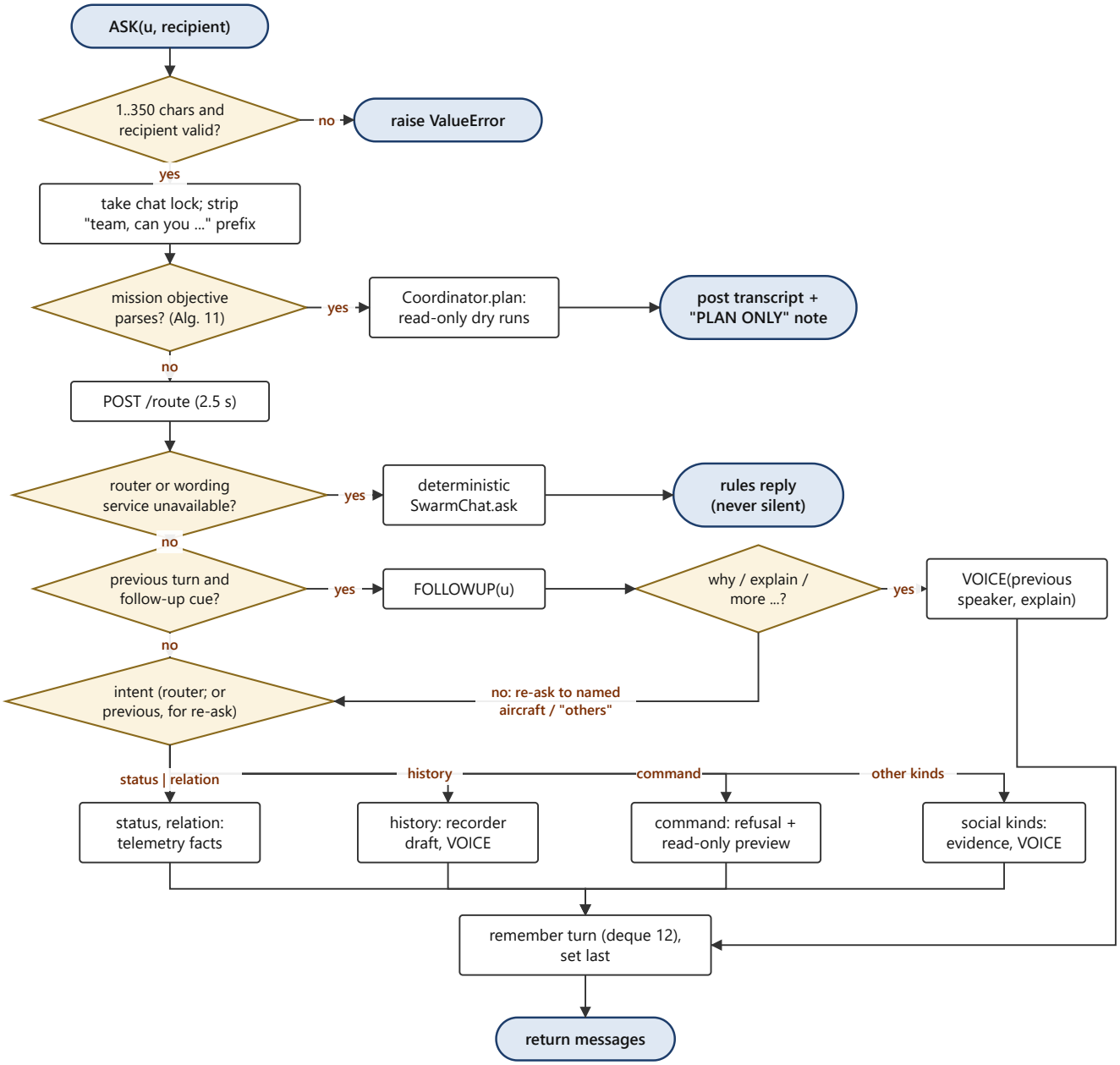



**Figure 12.1** Flowchart of `HybridChat.ask`, `_followup` and `_dispatch`. No branch reaches a flight endpoint; the command branch only previews how the mission parser would read the text. A re-ask with no resolvable addressee is dispatched as `uncertain` (social branch).

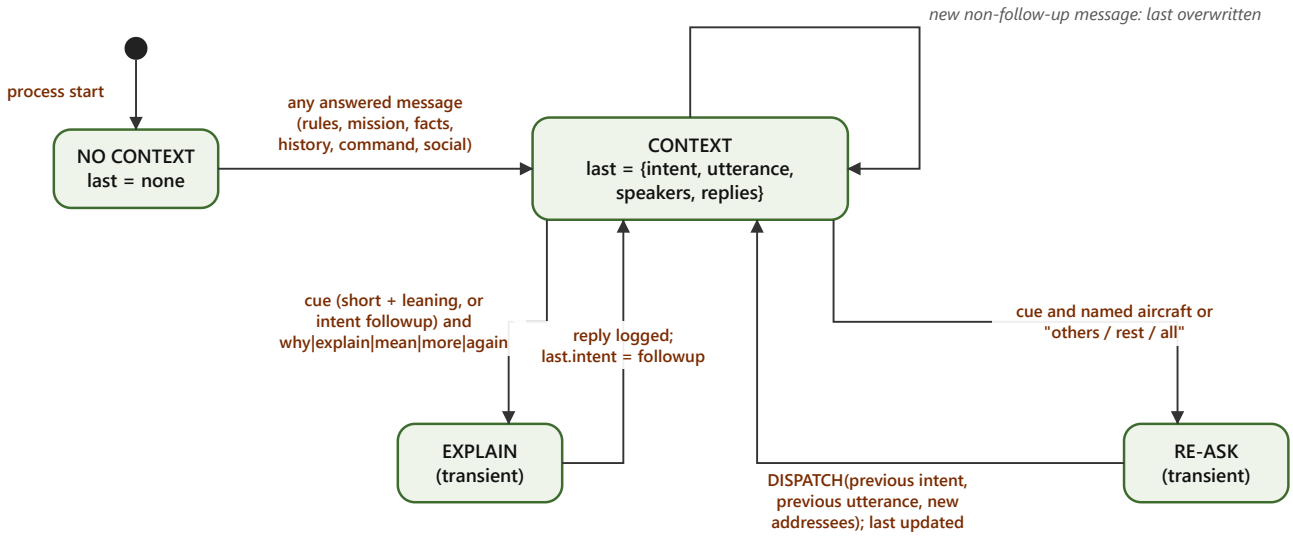



**Figure 12.2** Conversation follow-up state machine (`self.last`; `self.turns` keeps the last 12 turns, 8 of which are sent to the wording model). There is one such state per SpaceOps process, shared by all users; a restart returns to NO CONTEXT.

> **Reading the diagrams.** "Previous turn" means `self.last` is set; the follow-up cue is *intent = followup*, or (≤ 6 words *and* a leading *and / what about / how about / so / but / why / really / explain / tell me more / go on and* intent ∈ {uncertain, meta, followup, social}). The rules-engine branch and the mission branch also set `last` (intent `rules` or `mission`); a later re-ask of such a turn is dispatched as `uncertain`, because only the eleven router intents are carried over.

## 5. Parameters

| Parameter | Value |
|---|---|
| Router backbone / head | SmolLM2-135M frozen; z-scored features; one Linear layer; checkpoint `spaceops_v8/models/chat_intent_head.pt` (separate from HU's) |
| Router confidence floor | 0.55 (MIN_CONF) |
| Training data | `models/chat_intent_data.json`: 288 training texts (11 classes, 20–36 each), 47 held-out |
| Wording model | SmolLM2-360M-Instruct (fallback 135M-Instruct), offline, port 8767, 1 worker, queue 4 |
| Timeouts | /route 2.5 s; /generate 16 s client, 14 s job; health cached 5 s |
| Memory | 12 turns kept, 8 sent to the model; follow-up "short" = ≤ 6 words |

## 6. Complexity

Routing: one encoder pass (≤ 64 tokens) and an 11-way linear layer. Social/history replies: up to two generation calls of ≤ 60 new tokens each, serialised by a single worker. A mission objective triggers the coordinator's planning cost (Algorithm 11, ≤ 14 dry runs).

## 7. Properties

### Guaranteed

- **No actuation path:** the module and its HTTP route only append chat messages; the command intent produces a refusal and a preview; mission objectives are planned, never executed.
- **Graceful degradation:** if the router or the wording service is unavailable, answers come from the deterministic engine; if generated text is rejected, the deterministic fallback text is used and the rejection

reason is recorded.
- Status and relation answers are produced by the deterministic engine from telemetry, not by the language model.
- Every message carries a provenance label distinguishing learned wording from facts and rules.

### Not guaranteed

- Intent classification can be wrong; the confident-accuracy figure below is on a small held-out set written by the developers.
- Generated wording is constrained only by the validator (Algorithm 13); accepted text can still be awkward or vague.
- Follow-up detection is heuristic (word lists and length) and session state is global to the SpaceOps process (not per user).
- Mission-objective detection is attempted before routing, so an objective-like sentence always triggers a (read-only) planning call.

## 8. Testing and measured results

- Router held-out evaluation, from the checkpoint metadata logged at service start (`logs/chat_llm/*.jsonl`, event ROUTER_LOADED; trained 2026-11-11T05:43:11Z, seed 0): 47 test items, accuracy 0.872; accuracy when confident 0.917 at coverage 0.766 (threshold 0.55); command false positives 0, false negatives 0.
- Deterministic engine: `acceptance/tests/test_swarmchat.py` (18 tests, run by `run_swarmchat_tests.sh` / `run_chat_unit.sh`). The hybrid dispatch logic in `swarmchat_hybrid.py` has **no dedicated unit test** in `acceptance/tests`; it is exercised by `acceptance/chat_probe.py`, `human_chat.py` and `swarmchat_transcript.py` (no recorded results found). Results of the chat unit runners are **not recorded here**.

## 9. Relation to other algorithms

Shares the frozen encoder and pooling with Algorithm 1 (separate head and checkpoint). Uses Algorithm 13 for every generated sentence, Algorithm 17 for history answers, Algorithm 11 for PLAN-ONLY mission discussion, and Algorithm 3 for the read-only command preview. Messages from Algorithms 14 and 15 appear in the same chat stream with their own source labels.

## 10. Comparison with alternatives and related work

SwarmChat combines three ideas that exist separately in the literature: classifying an utterance into an intent before deciding how to answer, conditioning a generator on external evidence, and placing a programmable check between the generator and the user. The table compares the design with established representatives of each idea. The comparison is qualitative; none of the alternatives was run on the SwarmChat data.

| Approach | How it works | Where it is stronger than SwarmChat | Where SwarmChat differs |
|---|---|---|---|
| Retrieval-augmented generation (RAG) [1] | A retriever selects passages from a large corpus; a sequence-to-sequence generator conditions on them, end to end trained. | Open-domain coverage; retrieval over documents; learned joint training; widely evaluated on knowledge-intensive benchmarks. | No retrieval: the “evidence” is a small structured record built deterministically from live telemetry, plus an optional deterministic draft. The domain is closed (five aircraft, one arena). Generation is post-checked (Algorithm 13); RAG by itself does not guarantee that the output stays within the retrieved content. |

| | | | |
|---|---|---|---|
| Pipeline NLU + dialogue manager (e.g. Rasa NLU/Core) [2] | Intent classifier and entity extractor feed a trained or rule-based dialogue policy that selects the next action; tracker stores slots and history. | Entity/slot extraction, trained dialogue policies, form filling, mature tooling and per-user conversation trackers. | Same split of understanding and action selection, but SwarmChat has only an 11-way intent head on a frozen encoder, no entity model (names found by regular expressions), a hand-written dispatch table and a single global follow-up state (Figure 12.2) rather than per-user trackers. |
| Programmable guardrails (NeMo Guardrails) [3] | A runtime with a dialogue-flow language defines input, dialogue and output rails around an LLM; rails can call checks or block topics. | Declarative, reconfigurable rails; topic control; reusable across models and applications; documented evaluation. | The rails here are code, not configuration: the module has no flight endpoint at all (a structural rather than a policy guarantee), commands always get a refusal and a read-only preview, and every generated sentence passes the validator of Algorithm 13. Changing a rule requires a code change and re-test. |
| Template / data-to-text NLG with content planning [4] | A content plan selects which records to mention; a generator (neural or template) realises them. | Explicit content selection and ordering; learned fluency over many records (e.g. sports statistics). | Facts (status, relation, history drafts) are produced by templates in the deterministic engine; the language model only rephrases. Content selection is fixed per intent rather than learned. |

> **Where this design is weaker.** The router is a single linear layer trained on 288 developer-written sentences; its held-out set (47 items) was written by the same people, so its accuracy is optimistic for visitors. Follow-up detection relies on word lists and a 6-word length test and has no notion of who is asking. There is no retrieval over documentation, so architecture answers are limited to the six fixed statements in `ARCHITECTURE`. Compared with the systems above, there is no published evaluation, no per-user state and no configuration language for the rules.

**Honesty notes.** The wording model produces persona-styled text; it does not reason about the mission, and the personas are wording only (same controller, same facts). Messages labelled `TELEMETRY_FACT` come from templates, not from the model. Chat cannot actuate: the module contains no call to a flight endpoint, and mission objectives are planned read-only and must be approved and launched from the console.

## References


1. P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. https://arxiv.org/abs/2005.11401
2. T. Bocklisch, J. Faulkner, N. Pawlowski, A. Nichol, "Rasa: Open Source Language Understanding and Dialogue Management," *NIPS Workshop on Conversational AI*, 2017. https://arxiv.org/abs/1712.05181
3. T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, J. Cohen, "NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails," *Proc. EMNLP 2023: System Demonstrations*, pp. 431–445, 2023. https://aclanthology.org/2023.emnlp-demo.40
4. R. Puduppully, L. Dong, M. Lapata, "Data-to-Text Generation with Content Selection and Planning," *Proc. AAAI Conference on Artificial Intelligence* 33(01), pp. 6908–6915, 2019. https://doi.org/10.1609/aaai.v33i01.33016908


---

This specification describes the behaviour of the source revision identified above. Line-level references use `file:function`. Where the code and its own comments disagree, the code is described. Simulation-only system: no statement here is a claim about flight hardware.